

Keyboard Project

Table of Contents

Table of Contents.....	2
Notebooking Introduction.....	3
6/22/26 Design Matrix/Pros Cons List.....	4
Background Research.....	6
6/23/26 Research: CAD Selection & Setup.....	7
6/25/26 Research: CAD Selection & Setup.....	11
6/26/26 Skills: OnShape.....	14
6/28/26 Skills: KiCAD.....	15
6/29/26 Skills: Soldering.....	16
7/1/26 Research: PCB and Switches.....	18
7/4/26 Design: Picking a 3D Printer.....	20
Keyboard1.....	21
7/3/26 Design: PCB Day 1.....	22
7/3/26 Research: Keyboard Prototype.....	23
Appendix: Resources.....	24
References.....	25



Notebooking Introduction

6/22/26 Tools: Design Matrix/Pros Cons List

Designed By: Kirk Brian Ventura

Designed On: 6/22/26

Goal: Explain the concept of Design Matrix/Pros Cons list and describe the importance of effectively weighing options.

Design Matrices:

Definition: A structured tool that allows for design decisions to be compared systematically and quickly based on specific, weighted criteria. Our matrix will use columns for design decisions and rows for criteria.

	Decision 1			Decision 2		Decision 3	
Criteria	Weight	Score (1-10)	Weighted Score	Score (1-10)	Weighted Score	Score (1-10)	Weighted Score
1	9	10	90	6	54	2	18
2	8	9	72	7	56	3	24
3	6	8	48	9	43	6	36
Total			210		153		78

Importance:

- Comprehensive comparison
 - Side-by-side comparison of different design ideas
 - Allows for effective weighing of options against important criteria
- Objective Evaluation
 - Reduces biases in decision-making through weighing against the same criteria
 - Ensures decisions are based on quantifiable data
- Identifying Trade-offs
 - They help identify trade-offs. For example, a design could be easier to build but less efficient, while another is harder to build but more efficient.

Pros and Cons List:

Definition: A pros and cons list is a decision-making tool used to evaluate the positive and negative factors of a particular choice or option. It is straightforward but less structured than a decision matrix. Nevertheless, it is valuable in certain situations.

Importance:

- Simplified Decision Making
 - A pros and Cons list simplifies the decision-making process by breaking down design options into advantages and disadvantages.
 - A straightforward approach is easier to grasp.
- Flexibility
 - Highly flexible and can be used in various contexts without the need for detailed criteria and quantitative data.
 - A practical tool that is easy to use for quick, simple analysis of different options

EXAMPLE DESIGN OPTION	
Description	Describe the design decision
Pros	● Big Pro 1 ● Big Pro 2 ● Pro 1 ● Pro 2 ● <i>Small Pro 1</i> ● <i>Small Pro 2</i>
Cons	● Big Pro 1 ● Big Pro 2 ● Pro 1 ● Pro 2 ● <i>Small Pro 1</i> ● <i>Small Pro 2</i>

Conclusion:

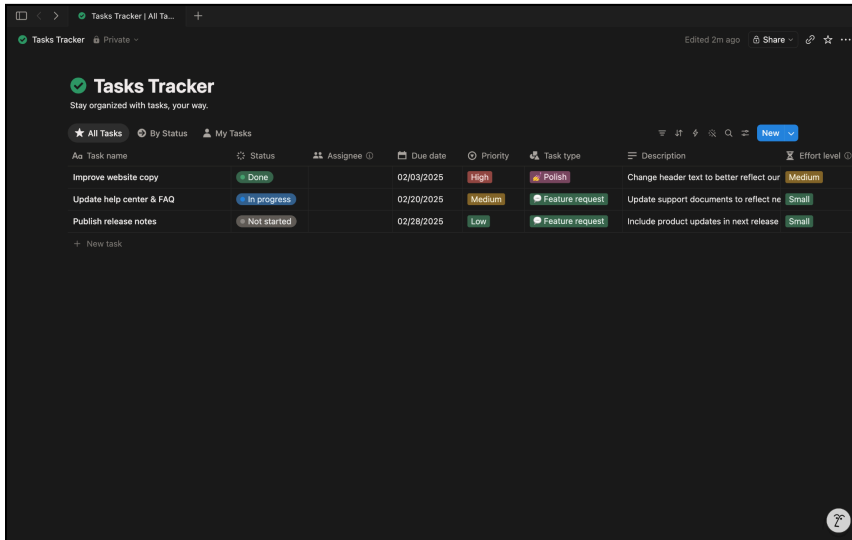
- Both the Pros/Cons list and the Design decision matrices will play an important role in my design process
- Design Matrix
 - Structured and Objective
 - Many Options
 - Detailed
- Pros and Cons
 - Flexible
 - Simple
 - Effective

6/22/26 Tools: Notion

Designed By: Kirk Brian Ventura

Designed On: 6/22/26

Goal: Explain the concept of Notion and how I'm using it for this project.



To manage the scope and complexity of this project, I set up a task tracker in Notion to maintain awareness of what needs to be done, what is currently in progress, and what has already been completed. A project like building a custom keyboard from scratch spans many parallel workstreams—PCB design in KiCad, case design in OnShape, firmware configuration, component sourcing, soldering, and documentation—and without a centralized place to track tasks, it becomes easy to lose track of

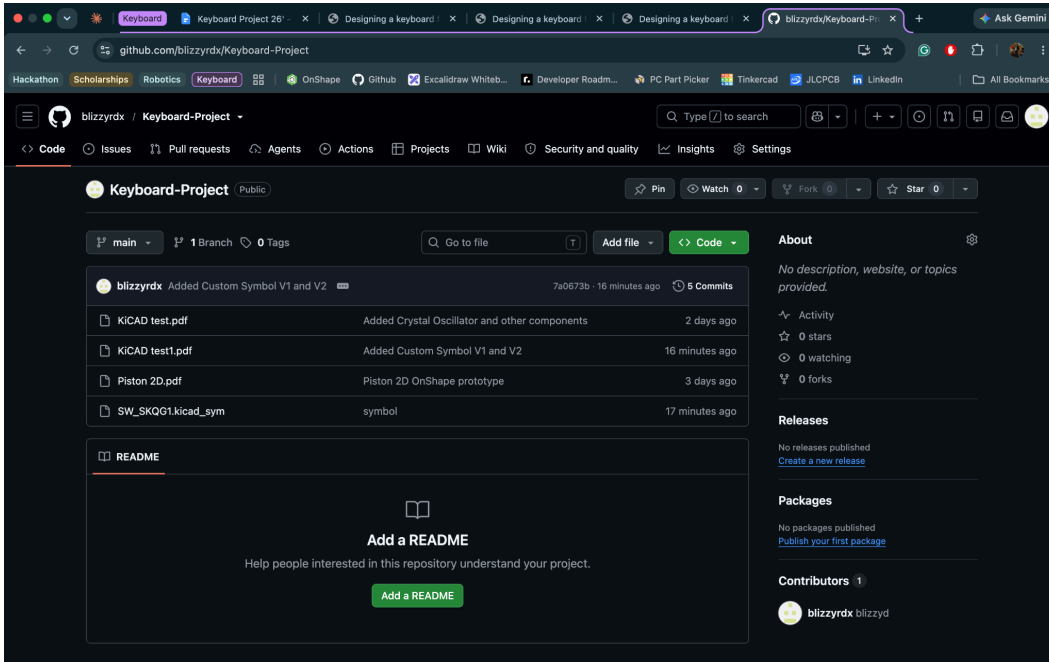
dependencies, forget steps, or work on the wrong thing at the wrong time. Notion's database view allows me to log each task with a status (Not Started, In Progress, or Done), a priority level (Low, Medium, or High), a task type tag (such as Design, Research, or Feature Request), a due date, and a description, giving me a clear at-a-glance picture of where the project stands at any given moment. This is particularly useful for a project that spans weeks or months and involves learning entirely new tools simultaneously — being able to open Notion and immediately see what the next actionable task is removes friction and keeps momentum going even when switching between very different types of work. Beyond simple task tracking, Notion also serves as a reference layer: notes from research sessions, links to datasheets, component specifications, and tutorial references can all be attached to relevant tasks so that information is findable in context rather than scattered across browser bookmarks or separate documents. Using a structured task management system like this reflects standard engineering project management practice, where maintaining visibility into project status is considered as important as the technical work itself.

7/10/26 Tools: Github

Designed By: Kirk Brian Ventura

Designed On: 7/10/26

Goal: Explain how I'm using GitHub to open source my keyboard project and track updates as I make them.



To keep my keyboard project organized and accessible, I created a public GitHub repository called Keyboard-Project where I store all my design files, including my KiCad schematics, custom symbol libraries, and CAD exports from OnShape. Making the repository public means anyone interested in the project — whether it's other keyboard builders, classmates, or scholarship reviewers — can view my

files, see how the design evolved, and even use my custom parts (like the SW_SKQG switch symbol) in their own projects. Each time I finish a meaningful chunk of work, like designing a new symbol or updating the schematic, I commit those changes with a short descriptive message (e.g. "Added Custom Symbol V1 and V2") so the commit history acts as a running log of my progress. This gives me a few advantages beyond just backup: I can look back at any previous version of a file if I make a mistake, I can track exactly how much progress I've made over time, and I'm building a portfolio-style record of the entire project that's transparent and easy for someone else to follow. Since GitHub also renders files like PDFs and schematics directly in the browser, viewers don't even need KiCad installed to see my work — they can just open the repo and look through it directly.



Background Research

6/23/26

Research: CAD Selection & Setup

Designed By: Kirk Brian Ventura

Designed On: 6/23/26

Goal: Explore different types of CAD software to use for this project.

A custom keyboard build actually requires two different kinds of CAD work: mechanical design (the case, plate, and physical structure) and electrical design (the PCB, switch matrix, and microcontroller wiring). No single piece of software handles both well, so I needed to choose one tool for each category.

Below is my analysis of each tool, including tradeoffs I considered before committing to this combination.

Onshape  onshape	Onshape is a free computer-aided design software for a wide array of use cases. Onshape is cloud-native software that provides a centralized repository, eliminating data loss and the need for users to manage their own files.	
	Pros: • Completely free for personal/non-commercial use • No installation • Automatic, granular version history • Cloud storage means accessible from any device with no manual file syncing • Strong documentation and active learning resources	Cons: • Requires Internet • Learning Curve • Faces lag issues with large files • No simulation

[Autodesk Fusion 360](#)



Fusion 360 is Autodesk's cloud-connected CAD platform for product design and mechanical engineering, combining parametric 3D modeling with built-in tools for simulation, rendering, and CAM (computer-aided manufacturing for CNC machining).

Pros:

- Free for hobbyists, students, and startups
- Combines CAD, simulation, and CAM (CNC toolpaths) in a single application
- Huge tutorial/community ecosystem

Cons:

- Requires installation and periodic updates
- Real-time multi-user collaboration is more limited
- The free tier has restrictions
- Version history/branching is less seamless

[AutoCAD](#)



AutoCAD

AutoCAD is Autodesk's flagship drafting and drawing software, and one of the oldest and most widely recognized names in the CAD industry. Unlike OnShape or Fusion 360, AutoCAD is primarily a 2D drafting tool, meaning it's designed for producing precise technical drawings, floor plans, schematics, and layouts rather than 3D solid models. While it does have some 3D capability, it is rarely used for mechanical part design or product engineering. It is the industry standard in architecture, civil engineering, and construction, where detailed 2D drawings and plans are the primary deliverable.

Pros:

- Industry standard in architecture, civil engineering, and construction
- Extremely precise 2D drafting tools, unmatched for technical drawing and documentation
- Very mature software with decades of stability and documentation

Cons:

- Primarily 2D
- Expensive for non-students
- Steeper learning curve for someone coming from modern parametric CAD tools
- No parametric modeling history

[SolidWorks](#)



SolidWorks is a professional-grade parametric 3D CAD platform developed by Dassault Systèmes. It is one of the most widely used mechanical engineering tools in industry. It is a desktop-only Windows application with an extremely deep feature set including advanced simulation (FEA/stress analysis), sheet metal design, large assembly management, and GD&T (geometric dimensioning and tolerancing) tools used in professional manufacturing drawings.

Pros:

- Extremely powerful and feature-rich — considered the gold standard for mechanical engineering in many industries
- Advanced simulation, stress analysis, and motion study tools built in
- Excellent for large, complex assemblies with many parts
- Huge library of third-party add-ons and integrations for specialized industries
- Free for students through the SolidWorks Student Edition or school licenses

Cons:

- Very expensive commercially—licensing costs thousands of dollars per year, making it inaccessible without a school or company license
- Windows only—no Mac support at all natively
- No cloud collaboration or real-time co-editing like OnShape
- Steeper learning curve than OnShape or Fusion 360 for beginners
- Desktop-only with no browser access, and file management is entirely manual

[KiCAD](#)



KiCad is a free, open-source software suite specifically for designing printed circuit boards (PCBs): schematic capture, PCB layout, and 3D viewing all in one program. Unlike Onshape, it's a desktop application and is purpose-built for electronics rather than general mechanical CAD.

Pros:

- Completely free and open-source
- Industry standard for hobbyist/open-source hardware
- Works offline
- Native PCB-specific tools (footprint libraries, design rule checking, Gerber export for manufacturing)

Cons:

- Steeper learning curve, especially for PCB layout/routing
- Desktop-only
- Manual version control (you'd need Git, not built-in like Onshape)
- UI/UX is considered less polished than some paid alternatives.

6/25/26

Research: CAD Selection & Setup

Designed By: Kirk Brian Ventura	Designed On: 6/25/26
---------------------------------	----------------------

Goal: Determine which CAD software to use for this project

Rather than arbitrarily selecting a CAD platform based on name recognition alone, I wanted to apply an engineering decision-making process to ensure I was choosing the right tool for this specific project. Engineering decisions, especially early ones like tool selection, have downstream consequences throughout a project, so making a deliberate, criteria-based choice is more reliable than choosing whatever seems most familiar or popular. To do this, I first researched the major CAD platforms available (covered in the software profiles above), then identified a set of criteria most relevant to my situation as a beginner building a keyboard case: accessibility and cost, ease of learning, collaboration and version control, suitability for mechanical part design, 3D print export compatibility, and platform availability on Mac. Each criterion was then assigned a weight reflecting its importance to this project specifically. For example, platform availability was weighted heavily because SolidWorks having no Mac support would be an immediate dealbreaker regardless of its other strengths. I then scored each CAD option against every criterion and calculated a weighted total, producing a decision matrix that made the tradeoffs between platforms visible and comparable at a glance. This process is documented below.

Criteria:

1. Accessibility / Cost

Free vs. paid, student licensing, whether it requires a powerful local machine or just a browser.

2. Ease of Learning / Beginner Friendliness

How quickly can someone with no CAD background become productive?

3. Suitability for Keyboard/Mechanical Part Design

How well does the tool handle the specific geometry you need, like precise rectangular cutouts, standoff holes, chamfers, and tight tolerances for switch plates??

4. Export & 3D Print Compatibility

How cleanly does the tool export STL/3MF for slicing?

5. Platform Availability

Windows only? Mac compatible? Browser-based?

	OnShape			Fusion360		Solid Works	
Criteria	Weight	Score (1-10)	Weighted Score	Score (1-10)	Weighted Score	Score (1-10)	Weighted Score
Accessability	9	10	90	6	54	2	18
Ease of Learning	8	9	72	7	56	3	24
Suitability	7	8	56	8	56	9	63
3D Print	6	8	48	9	43	6	36
Availablity	10	10	100	9	90	1	10
Total			366		310		151

Summary -

The decision matrix clearly identifies OnShape as the most suitable CAD platform for this project, achieving a weighted total of 366 compared to Fusion 360's 310 and SolidWorks' 151. Platform availability was the highest-weighted criterion at 10, reflecting the hard constraint of working on a Mac, a factor that effectively eliminated SolidWorks from serious consideration, since it has no native Mac support and would require running a Windows virtual machine just to launch the software. Accessibility and cost, weighted at 9, further reinforced OnShape's lead: it is entirely free with no installation required, no license to renew, and no feature restrictions for a project at this scale. SolidWorks, while scoring highest on raw suitability for mechanical design, was dragged down decisively by these two criteria alone. Fusion 360 was a genuinely competitive second-place option, particularly its slight edge in 3D print workflow compatibility, and remains a strong candidate for future projects like a robotic arm where its simulation, CAM, and generative design tools become directly relevant. For this specific project, however, OnShape's combination of zero cost, full Mac compatibility, and strong beginner accessibility made it the clear and well-justified choice.

Unlike the CAD software selection, which required a formal decision matrix to weigh several genuinely competitive options, the choice of PCB design software was straightforward enough that a separate matrix was unnecessary. KiCad is the clear and essentially universal choice for hobbyist and open-source PCB design—it is completely free and open-source with no licensing restrictions, it is the industry standard specifically within the DIY electronics and mechanical keyboard community, and it includes every tool needed for this project natively in one application: schematic capture, PCB layout, footprint libraries, design rule checking, and Gerber file export for manufacturing. The only real alternatives, such as Altium Designer or Cadence Allegro, are enterprise-grade tools that cost thousands of dollars per year in licensing fees and are designed for large professional engineering teams, not a student building a first keyboard. A decision matrix comparing KiCad to

Altium would be the equivalent of comparing OnShape to software designed for Boeing's engineering department: technically valid, but not a meaningful or realistic comparison given the constraints of this project. For PCB design at this scale and budget, KiCad is not just the best option, it is the only practical one, which made a formal weighted comparison unnecessary.

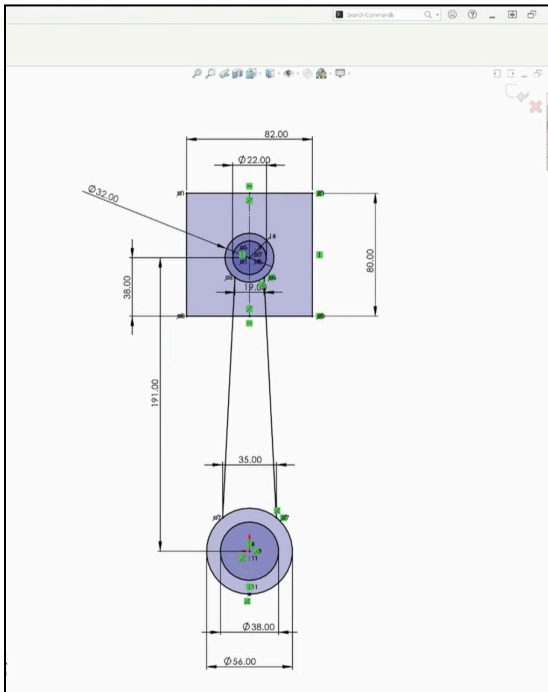
6/26/26 Skills: OnShape

Designed By: Kirk Brian Ventura

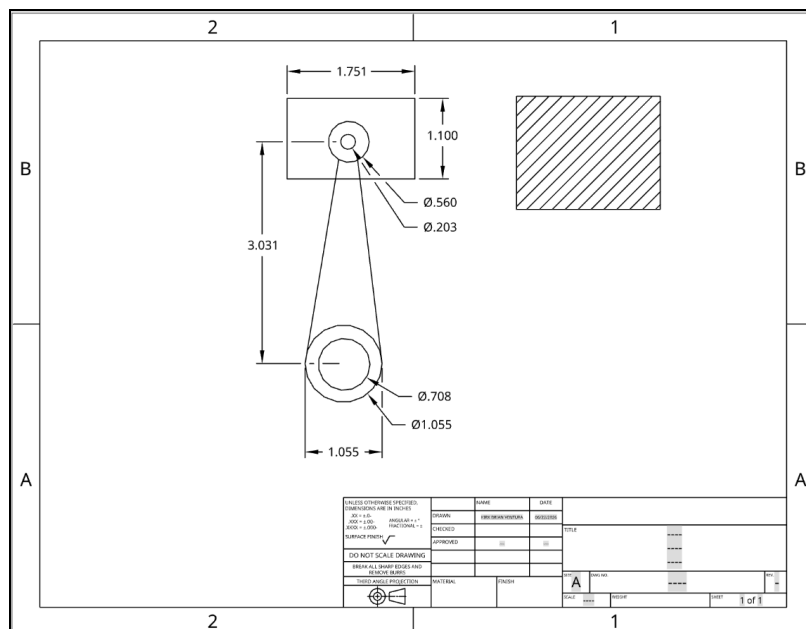
Designed On: 6/26/26

Goal: Familiarize myself with the OnShape interface.

Prior to this project, I had exposure to CAD only indirectly. I followed along on a robotics build where a teammate handled the Onshape modeling, but I hadn't designed anything myself. This section documents how I built the skills from scratch, specifically to execute this project.



While working on this drawing, I learned the basics of OnShape's 2D drafting tools in the Drawing workspace. I practiced inserting orthographic views of my piston part, including front and section views, and used the dimension tool to label key measurements like diameters ($\varnothing.708$, $\varnothing.1.055$, $\varnothing.560$, $\varnothing.203$) and linear distances (1.100, 1.751, 3.031, 1.055). I also got familiar with how OnShape automatically generates extension and dimension lines once you select an edge or circle, and how to reposition those lines for a cleaner layout. Additionally, I learned how section views work, using the cutting plane indicators (A-A and B-B) to show internal features of the part that wouldn't otherwise be visible from the outside. Overall, this part of the project helped me understand how a 3D model translates into a standard engineering drawing with proper dimensioning and projection conventions.



6/28/26

Skills: KiCAD

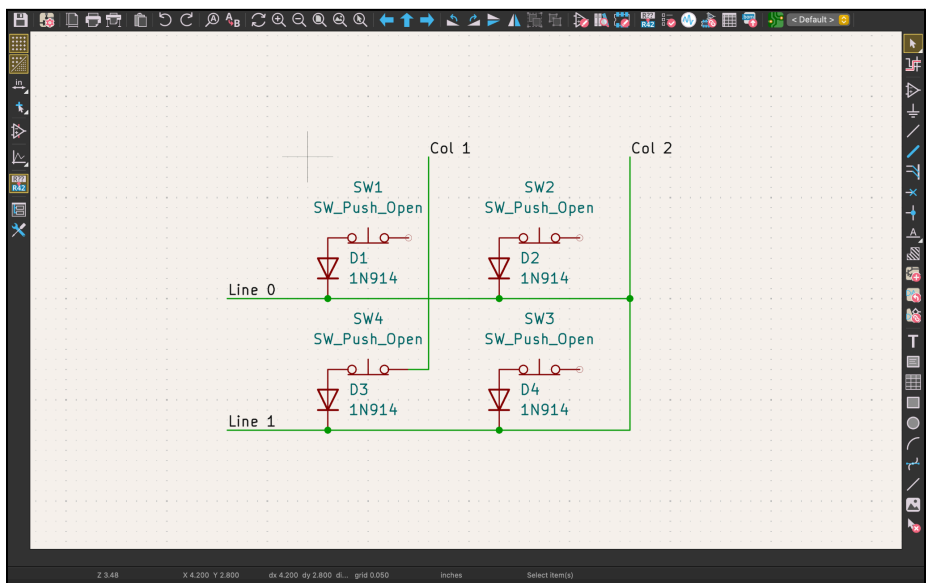
Designed By: Kirk Brian Ventura

Designed On: 6/28/26

Goal: Familiarize myself with the KiCAD software and learn to design simple schematics and PCBs, building toward more complex designs later in this project.

Before starting this project, I had no prior experience with KiCAD or any PCB design software. However, I came into this with a solid theoretical foundation in circuits from AP Physics C: Electricity and Magnetism, where I studied concepts like Kirchhoff's laws, RC/RL circuits, and basic circuit analysis. I had also built simple circuits before on breadboards, but I had never translated a circuit into a formal schematic or designed a physical PCB layout. This project allowed me to bridge that gap between circuit theory and practical hardware design: learning not just how to draw a schematic, but how that schematic becomes a routed, physical board that can actually be manufactured.

Using this schematic, I learned the fundamentals of schematic capture in KiCad's Schematic Editor. I practiced placing and wiring components, including push-button switches (SW1–SW4) and 1N914 diodes (D1–D4), and connected them using the wire tool to form a basic 2x2 button matrix. I learned how to use labels (such as Col 1, Col 2, Line 0, and Line 1) to name and organize nets, which makes the schematic easier to read and helps KiCad recognize



logically connected points even when wires aren't physically touching on the canvas. I also got familiar with the component placement workflow: searching the library for parts like SW_Push_Open and 1N914, dropping them onto the sheet, and rotating/aligning them to keep the wiring clean. Additionally, I learned why diodes are necessary in a button matrix: they prevent "ghosting," where current could otherwise flow backward through unintended paths when multiple keys are pressed at once, which is a critical concept for any keyboard matrix design.

6/29/26

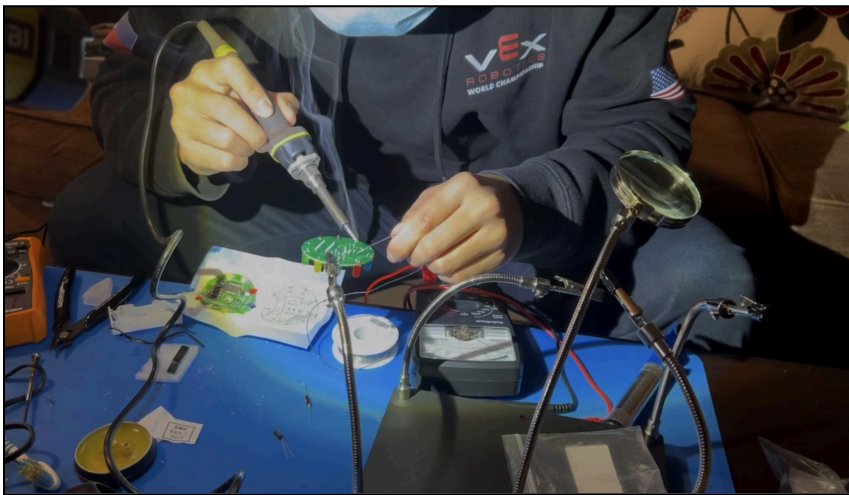
Skills: Soldering

Designed By: Kirk Brian Ventura

Designed On: 6/29/26

Goal: Learn the fundamentals of through-hole soldering, including proper tool use, technique, and safety as a foundational hand skill for assembling the keyboard's PCB. This section covers through-hole (THT) soldering specifically.

Before this project, I had little to no hands-on soldering experience. This was my first real opportunity to practice the physical assembly side of electronics, not just the design side (OnShape, KiCad).



What I Learned -

Through-hole soldering is the process of attaching a component to a PCB by inserting its metal leads through pre-drilled holes and melting solder around the joint on the opposite side, creating a permanent electrical and mechanical connection. Below are the tools and materials I learned to use, along with their function:

Tools & Materials

- **Soldering iron** — The main heating tool, with a metal tip that heats up (typically 300–370°C / 570–700°F depending on the solder and component) to melt solder onto the joint. I learned that controlling iron temperature matters: too cold and the solder won't flow properly (a "cold joint"), too hot and you risk damaging components or lifting pads off the PCB.
- **Soldering iron stand** — Holds the iron safely when not in use, since the tip stays extremely hot and can cause burns or fires if set down on the table.
- **Solder wire** — A thin metal wire (commonly a tin-lead or lead-free alloy) that melts at a low temperature and solidifies to form the electrical connection between the component lead and the copper pad. I learned solder often contains a flux core, which helps the joint flow smoothly and bond properly.

- **Flux** — A chemical that cleans the metal surfaces and removes oxidation right before the solder melts, allowing it to flow evenly and bond to both the component lead and the pad. Without flux, joints can come out dull, weak, or "cold" (improperly bonded). Some solder wire has a flux core built in, while extra flux can be applied separately for tricky joints.
- **Damp sponge** — Used to wipe and clean the soldering iron tip between joints. Over time, the tip oxidizes and picks up burnt flux residue, which prevents it from heating evenly. Wiping it on a damp sponge keeps the tip clean, shiny, and able to transfer heat efficiently.
- **Solder wick/desoldering braid** — A flat copper braid used to remove excess or unwanted solder, often for fixing mistakes or removing a component. I learned to lay the braid over solder, then heat it through with the iron so the braid wicks (absorbs) the melted solder away.
- **Tweezers** — Used to hold small components steady while soldering, since some leads are too small or hot to handle with fingers, and to avoid accidentally burning yourself.
- **Helping hands/fixture** — Holds the PCB or wires in place hands-free, which is especially useful since soldering generally requires both hands (one for the iron, one for the solder).
- **Isopropyl alcohol (IPA)** — Used after soldering to clean off leftover flux residue from the board, which can otherwise look messy or, in some cases, attract dust/moisture over time. I applied it with a brush or cotton swab to gently scrub the joints clean.
- **Brush (flux brush/cleaning brush)** — Used alongside the isopropyl alcohol to scrub away flux residue and any debris from the board after soldering.
- **Safety glasses** — Worn to protect the eyes from solder splatter, fumes, or flying debris when clipping leads.
- **Wire cutters/flush cutters** — Used to trim excess component leads after soldering, so they don't stick out and risk shorting against other components or the case.

Technique I Practiced

I learned the general through-hole soldering process:

1. Insert the component leads through the correct holes on the PCB, often bending them slightly to hold the component in place.
2. Heat both the pad and the lead simultaneously with the iron tip for roughly 1–2 seconds.
3. Feed solder into the joint (not directly onto the iron tip) so it melts and flows around the lead and pad, forming a clean, shiny, cone-shaped joint.
4. Remove the solder, then the iron, and let the joint cool and solidify without moving the component (movement during cooling causes a "cold joint," which is dull, cracked, and unreliable).
5. Clip excess lead length, then clean the area with isopropyl alcohol to remove flux residue.

7/1/26

Research: PCB and Switches

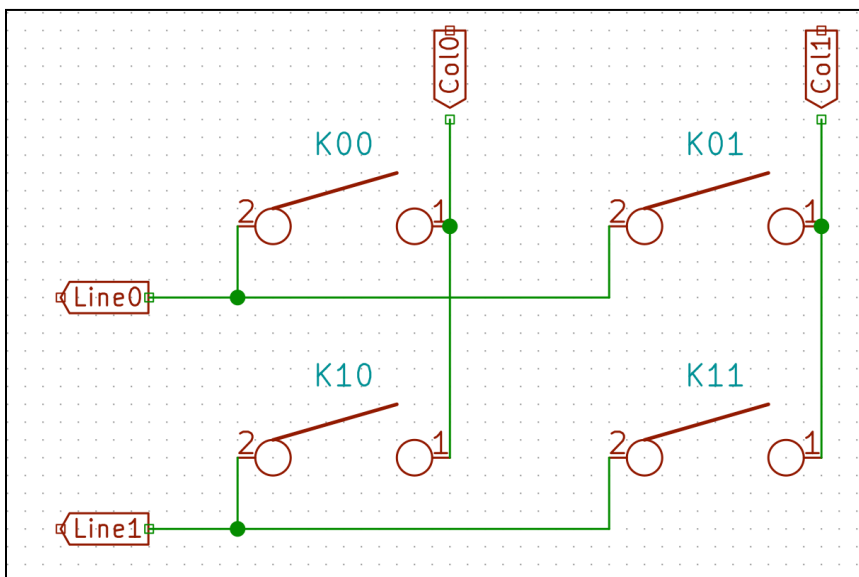
Designed By: Kirk Brian Ventura

Designed On: 7/1/26

Goal: Understand how a keyboard PCB electrically detects key presses and learn to design a switch matrix in KiCad, including the role of diodes in preventing ghosting, as the foundation for the keyboard's circuitry.

The PCB is the circuit board responsible for converting key presses into commands the computer can understand: specifically, the HID protocol (keyboard protocol) used over USB. It contains a microcontroller (MCU, which integrates a CPU, a small amount of RAM, flash memory, and numerous I/O pins) along with an array of switches arranged in a matrix.

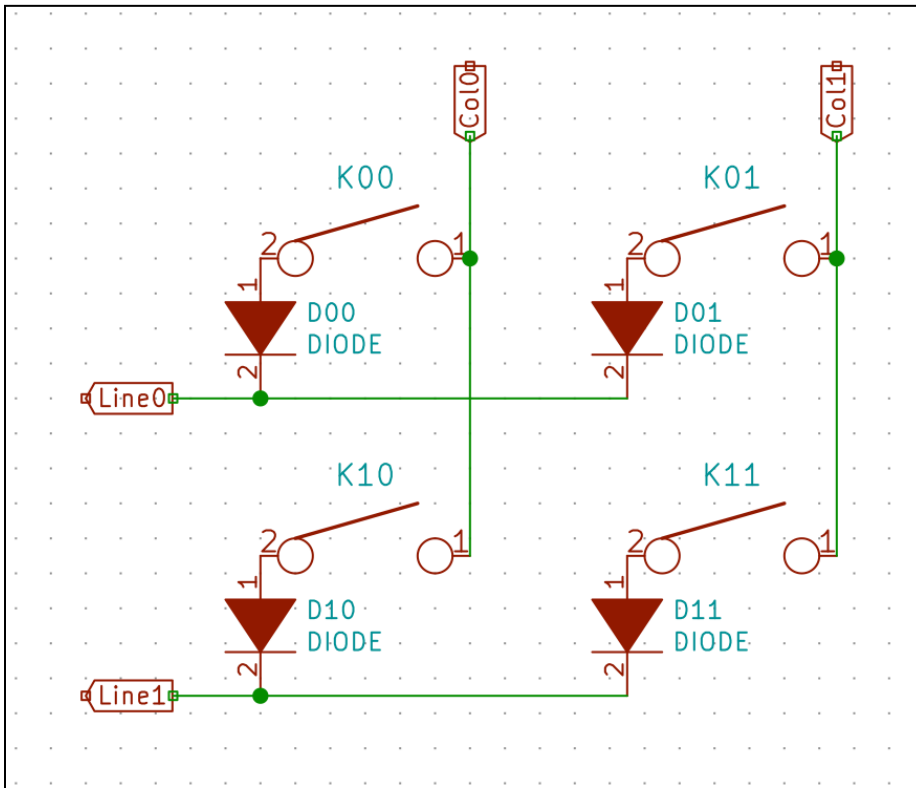
The switches are organized into rows and columns that mirror the physical layout of the keys. The MCU continuously scans this matrix at high speed: it applies voltage to one column at a time, then reads the state of each row before moving to the next column.



When a key is pressed, it closes the connection between its row and column, allowing current to flow from the energized column into that row. While scanning, the MCU detects this voltage change and—by tracking which column it was driving and which row picked up the signal—determines exactly which switch was activated. Once identified, the MCU translates the key press into a standardized key code and transmits it to the computer over the USB data lines.

When the controller powers Col0, and if K00, K01, and K11 are pressed simultaneously, the controller will see a tension of both Line 0 and Line 1 because the current from Col0 will flow to K00 pin 1, the pin 2 because the switch is closed, then to K01, and so on to K10. In essence, this means all four keys are pressed in the eyes of the microcontroller.

To prevent this, we can place diodes between the wires from each line and the switch. Diodes are electronic components that only allow current to flow in one direction.



This is the resulting matrix:

Note -

The triangle of a diode points in the direction in which conventional current flows. If current were to flow from Line 0 to Line 2 of the D00 Diode, it would be blocked.

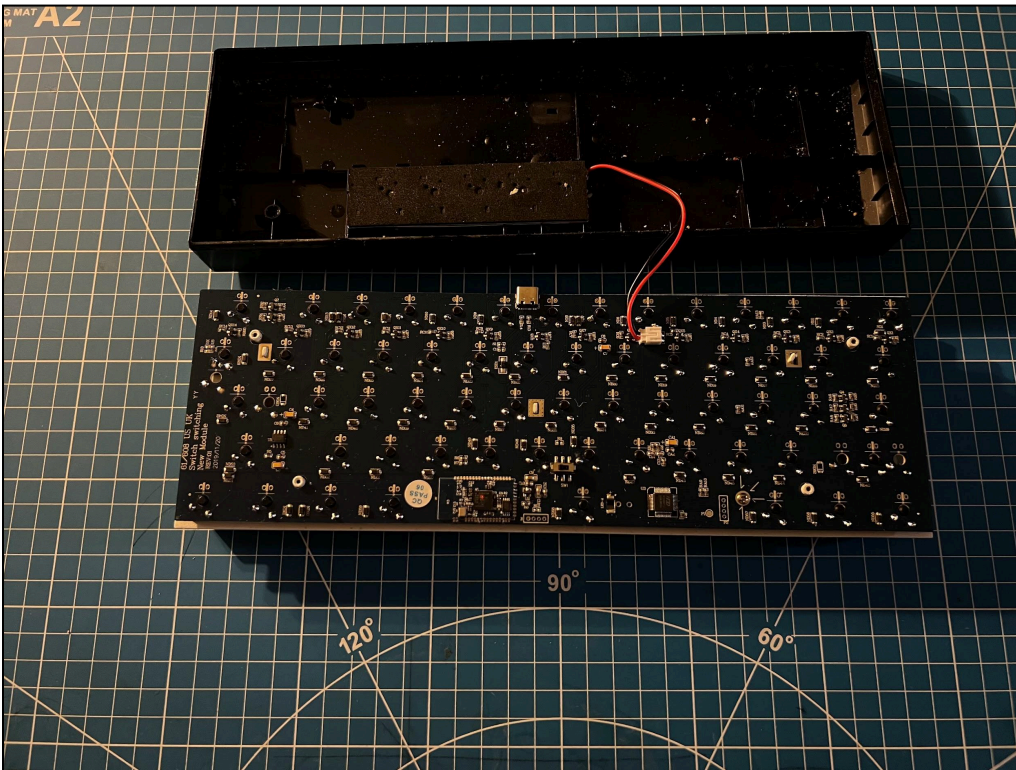
7/3/26

Research: RK61 Keyboard

Designed By: Kirk Brian Ventura

Date: 7/3/26

Goal: Research and reverse-engineer a commercially available keyboard (RK61) by disassembling it to understand how a real production keyboard is physically constructed.



Previously, I had an RK61 60% keyboard, which I disassembled piece by piece in order to study how a real, production-level keyboard is built from the inside out. Rather than going into this project with only theoretical knowledge from research and tutorials, I wanted to physically examine an existing keyboard to understand how its components relate to each other in practice—how the PCB sits in the case, how the switches mount and interface with the board, and what the soldering on a mass-produced

board actually looks like up close. Disassembling a real keyboard before designing my own gave me a concrete reference point: I could see exactly how switch pins align with PCB pads, how the controller and supporting components are laid out on the board, and how tightly tolerated the physical fit between the switch plate, PCB, and case actually needs to be.



Keyboard

1

7/3/26

Design: PCB Day 1

Designed By: Kirk Brian Ventura

Date: 7/3/26

To begin, I wanted to decide on a microcontroller first. Given the sheer number of microcontrollers on the market, I wanted to choose a user-friendly yet adequate first chip.

	ATmega32U4			RP2040		ESP32	
Criteria	Weight	Score (1-10)	Weighted Score	Score (1-10)	Weighted Score	Score (1-10)	Weighted Score
Native USB Support	10	10	100	10	100	3	30
QMK Compatibility	9	10	90	7	63	4	36
Cost	7	8	56	9	63	9	63
Beginner Friendliness	8	9	72	7	56	6	48
Hand-Solderable	8	9	72	8	64	7	56
Community Support	9	10	90	8	72	4	36
Total			480		418		269

The decision matrix identifies the **ATmega32U4** as the most suitable microcontroller for this project with a weighted total of 480, meaningfully ahead of the RP2040 at 418 and well ahead of the ESP32 at 269. The two highest-weighted criteria — native USB support and QMK firmware compatibility, both weighted 9 or 10 — were the decisive factors. The ATmega32U4 scored a perfect 10 on both, reflecting its decade-long track record as the keyboard community's chip of choice and the fact that QMK was effectively built around it. The RP2040 is a genuinely strong second-place option, and scores competitively across almost every criterion — its lower scores on QMK compatibility and beginner friendliness reflect the fact that its keyboard-specific ecosystem, while growing fast, is still maturing compared to the ATmega32U4's. The ESP32, despite being extremely cheap and widely available, was a poor fit for this specific application: its lack of native USB hardware means

implementing wired USB HID requires significant workarounds, and QMK does not officially support it, both of which would add major complexity to an already ambitious first build. It is worth noting that the ESP32 would score considerably higher in a matrix for a wireless Bluetooth keyboard, where its built-in Bluetooth hardware becomes a decisive advantage rather than an irrelevant feature. For a wired, QMK-based keyboard built by a first-time PCB designer, the ATmega32U4 is the clear and well-justified choice.

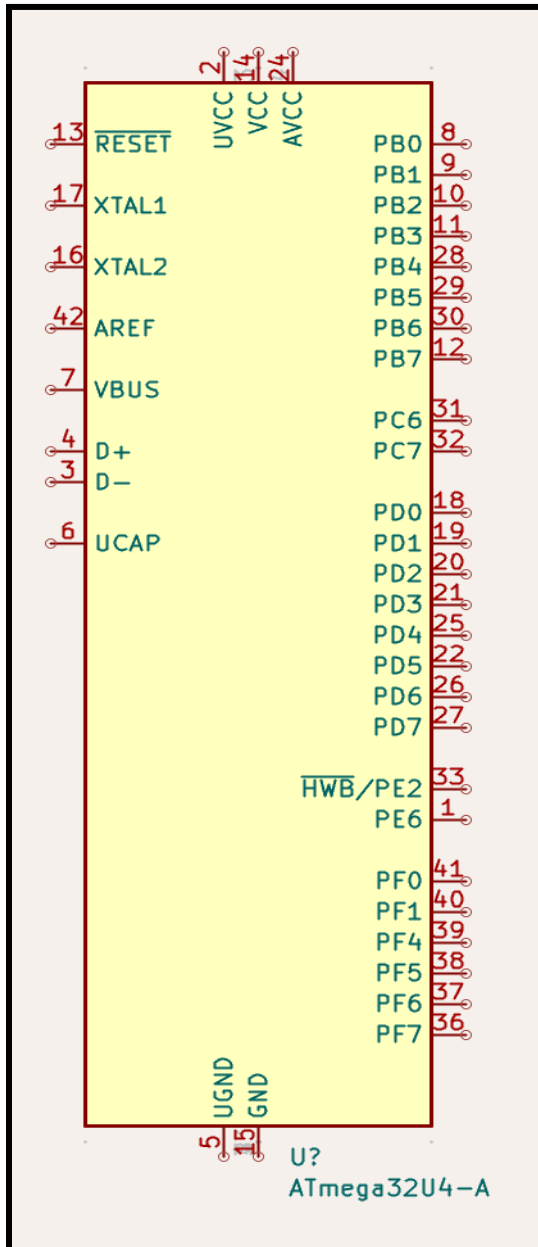
7/7/26

Research: Keyboard Schematic Day 1

Designed By: Kirk Brian Ventura

Date: 7/7/26

Goal: Research the microcontroller schematic and determine what each pin is used for.



ATmega32U4 Pin Reference: The ATmega32U4 is a popular AVR microcontroller with native USB support, the same chip used in the Arduino Leonardo, Arduino Micro, and Pro Micro boards, which makes it a natural fit for a USB keyboard controller. UVCC (pin 2), VCC (pin 14), and AVCC (pin 24) are all tied to +5V, with VCC powering the digital logic, UVCC powering the USB transceiver, and AVCC powering the analog-to-digital converter. UGND (pin 5) and GND (pin 15) are ground returns, split between USB ground and main logic ground. D+ (pin 4) and D- (pin 3) are the USB data lines that connect directly to the USB connector, while VBUS (pin 7) senses whether USB power is present. UCAP (pin 6) connects to the chip's internal USB regulator and requires a 1µF capacitor per the datasheet. RESET (pin 13) resets the chip when pulled low. XTAL1 (pin 17) and XTAL2 (pin 16) connect to an external crystal oscillator for precise clock timing, especially important for USB. AREF (pin 42) sets the ADC reference voltage but isn't needed in this design, so it's marked with an X. HWB/PE2 (pin 33) is the Hardware Boot pin, pulled down through a 10k resistor to force normal boot mode, and PE6 (pin 1) is unused, also marked with an X. The remaining pins are general-purpose I/O split across Port B (PB0–PB7, pins 8–12 and 28–30), Port C (PC6–PC7, pins 31–32), Port D (PD0–PD7, pins 18–27), and Port F (PF0–PF7, pins 36–41), available for wiring the key matrix.

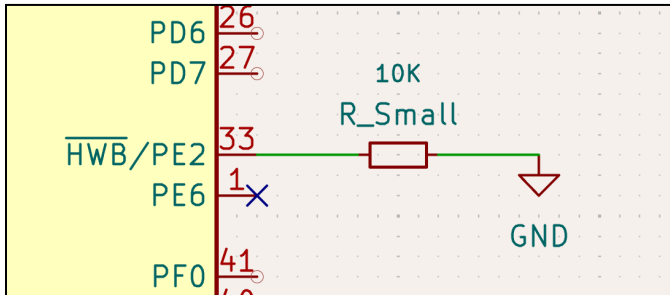
7/7/26

Research: Keyboard Schematic Day 1

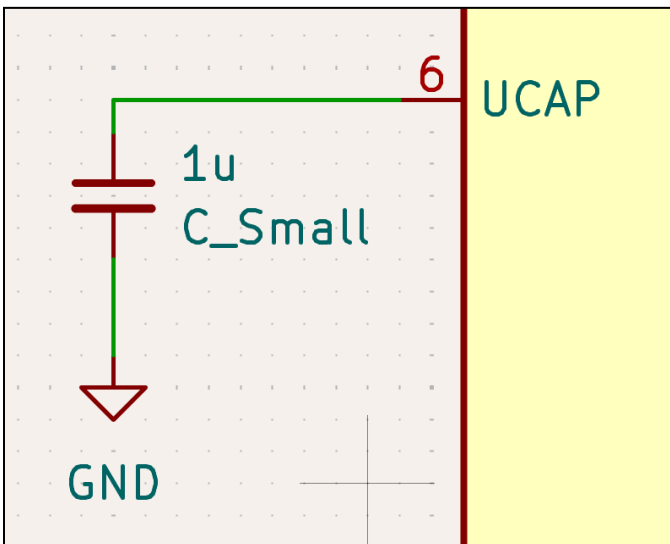
Designed By: Kirk Brian Ventura

Date: 7/7/26

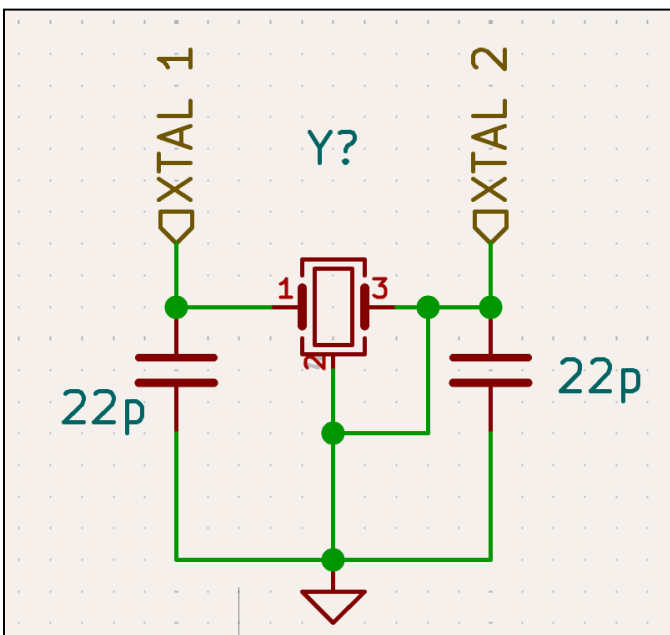
Goal: Research the microcontroller schematic and determine what each pin is used for.



HWB Pull-Down Resistor: HWB tells the chip whether to boot into the USB bootloader or run normal firmware. An unconnected pin floats and can pick up electrical noise, causing unpredictable readings, so a 10kΩ resistor pulls it down to GND to ensure the chip reliably boots into normal firmware mode. 10k is a common middle-ground value, strong enough to hold the logic-low state without interfering when something else needs to drive the pin high.

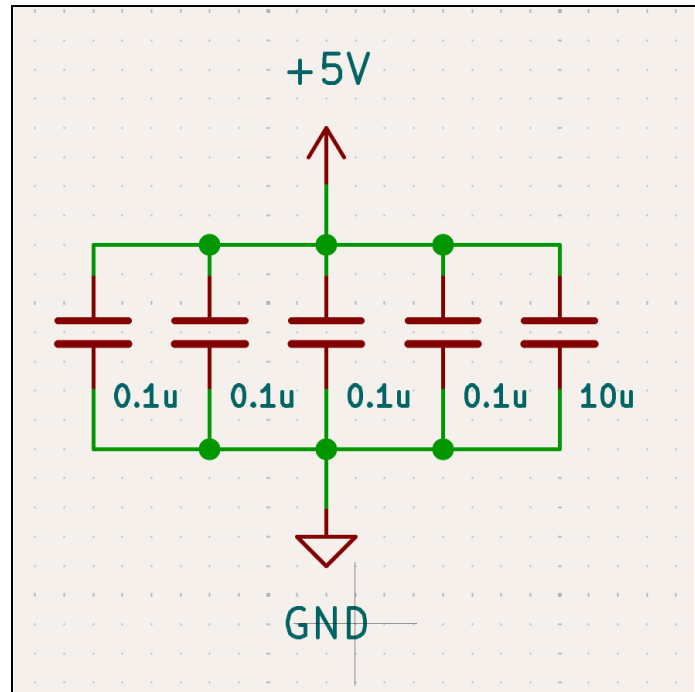


UCAP Capacitor: UCAP connects to the chip's internal USB voltage regulator, which needs a capacitor for stability and ripple filtering. Without it, the output can become noisy enough to break USB signal timing. The datasheet specifies exactly 1μF, a mandatory value rather than a design choice, and skipping it is a common cause of USB simply not working.

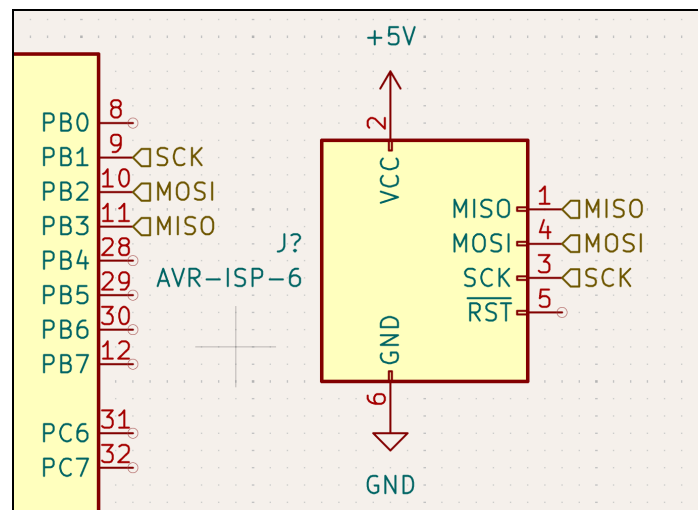


Crystal Oscillator (Pierce Oscillator Circuit): A crystal oscillator gives the chip a precise, stable clock signal. The ATmega32U4 has an internal oscillator, but it isn't accurate enough for USB communication, which requires tight timing tolerances. The crystal sits between XTAL1 and XTAL2, forming a feedback loop with the chip's internal amplifier circuitry since the crystal can't oscillate on its own. Two 22pF load capacitors, one on each XTAL pin, keep the oscillation balanced and reliable at startup, and both share a common GND to keep the ground return clean right at the crystal. This layout is called a Pierce oscillator.

Power Decoupling: The datasheet recommends a decoupling capacitor on every +5V pin of the MCU, since a sudden current draw from the chip can dip the supply voltage and create noise for other components sharing the line. Decoupling capacitors act as local energy storage, supplying that sudden current and recharging between uses. AN2519, Microchip's hardware design app note, specifies 100nF (0.1μF) per VCC-type pin. The ATmega32U4 has four VCC-type pins plus UVCC/VBUS, so in practice a single shared 10μF capacitor covers VBUS/UVCC while four 0.1μF capacitors cover the rest. All of these can be grouped in one cluster on the schematic since every VCC-type pin ties to the same +5V net, though on the actual PCB layout each capacitor should be placed physically close to the pin it's decoupling.



AVR-ISP-6 Header: This is a 6-pin In-System Programming (ISP) header that lets me flash firmware directly onto the ATmega32U4 using a dedicated programmer, bypassing USB entirely. It's necessary because a brand-new chip has no bootloader yet, so ISP is how the bootloader itself first gets burned onto the chip, and it also serves as recovery path if the firmware or bootloader ever gets corrupted. The header connects VCC and GND for power, along with MISO, MOSI, and SCK, which are the SPI communication lines used to transfer data between the programmer and the chip, plus RESET, which the programmer pulls low to put the chip into programming mode. The pin order (1, 4, 3, 5, 2, 6) follows the standard 2x3 layout used by nearly every AVR ISP programmer cable, so wiring it this way means any standard ISP programmer will plug in correctly without custom wiring. On the actual PCB, this would be a small 2x3 pin socket that I'd clip a programmer onto to load the initial bootloader before the keyboard can be used over USB.



a

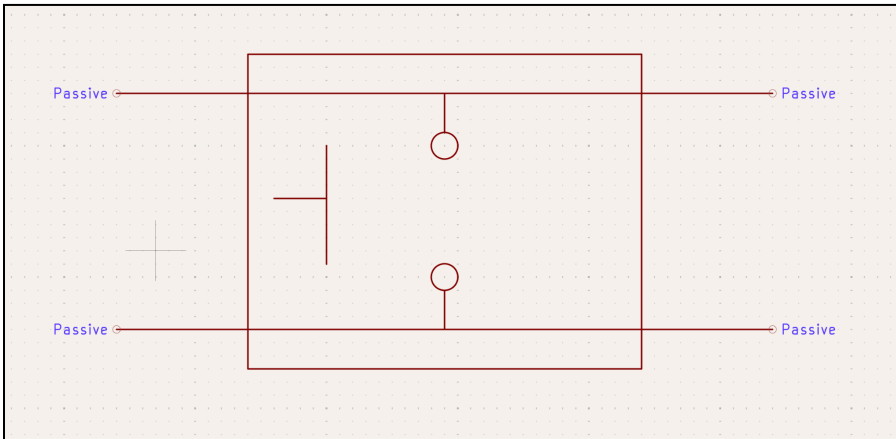
7/10/26

Research: Keyboard Schematic

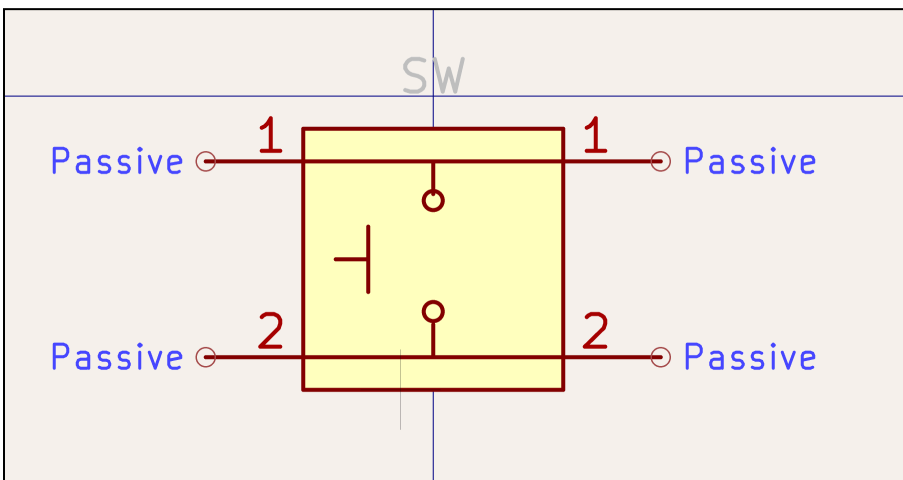
Designed By: Kirk Brian Ventura

Date: 7/10/26

Goal: Design and build a custom switch symbol in KiCad instead of using a pre-made library part.



I started by opening KiCad's Symbol Editor and creating a new symbol from the built-in Switch library as a base, so I could see how existing symbols were structured before building my own from scratch. This let me study things like pin placement, spacing, and how existing parts used basic shapes to represent a switch.



From there, I built my own symbol using basic drawing tools — a rectangle for the body, lines for the switch contacts, and circles to represent the actuator points. Each shape had to be placed carefully so the pins lined up on the schematic's grid, otherwise wires wouldn't connect properly once the symbol was placed on the schematic.

Once the shapes were drawn, I went into each element's properties to fine-tune things like line width, circle radius, and exact X/Y coordinates, since dragging shapes by hand didn't always give precise or grid-aligned results. This was especially important for getting the circle small enough and positioned correctly relative to the lines. While working through pin properties, I also discovered concepts like **Passive** pin type, which is used for components like switches and resistors that don't actively drive a signal but just pass current through. Learning about pin types, electrical rules, and how KiCad checks these during ERC (Electrical Rules Check) helped me understand why certain pin settings matter beyond just visual appearance.

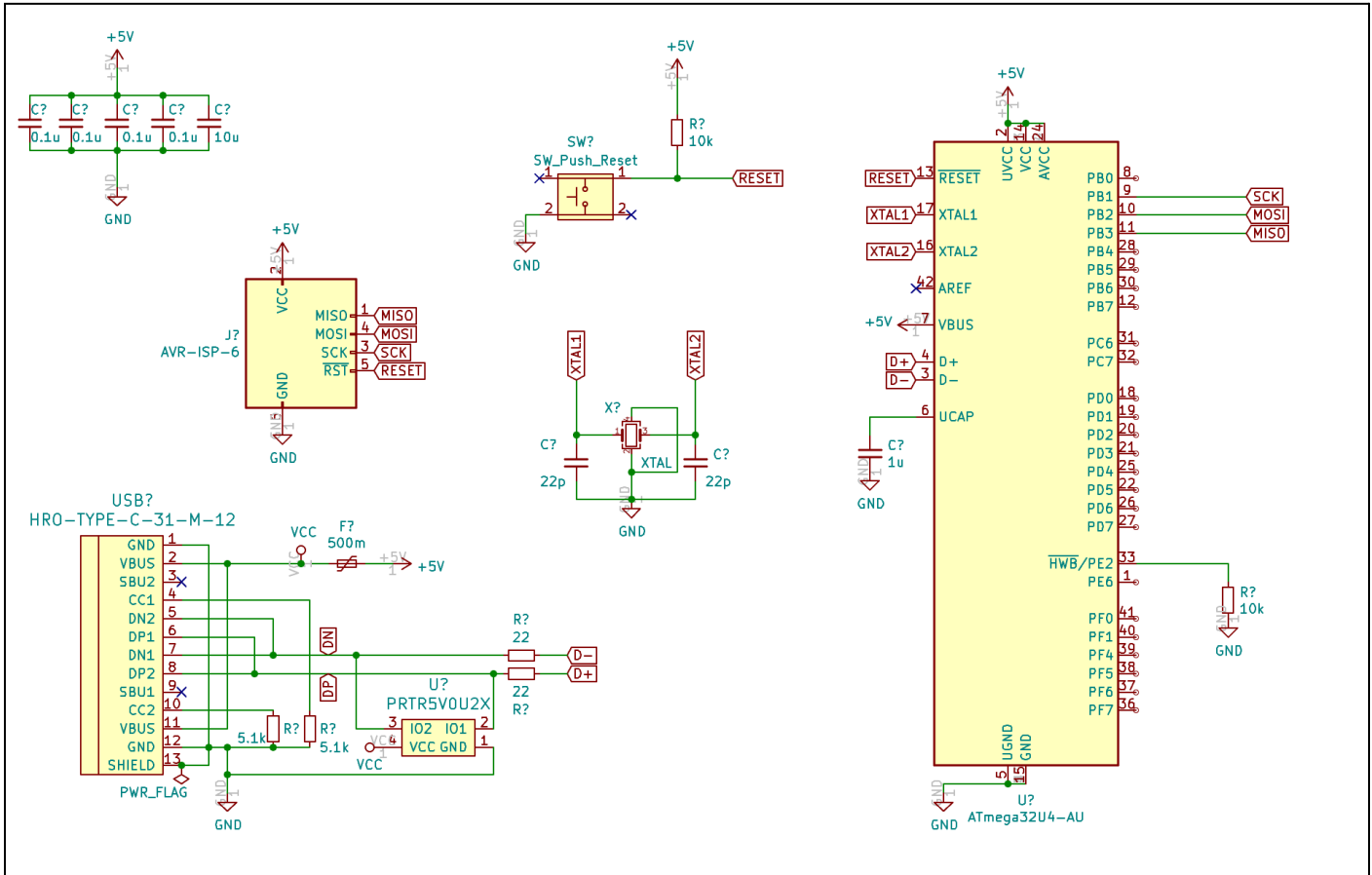
x/x/26

Research: Keyboard Schematic

Designed By: Kirk Brian Ventura

Date: 7/10/26

Goal: Design and build a custom switch symbol in KiCad instead of using a pre-made library part.



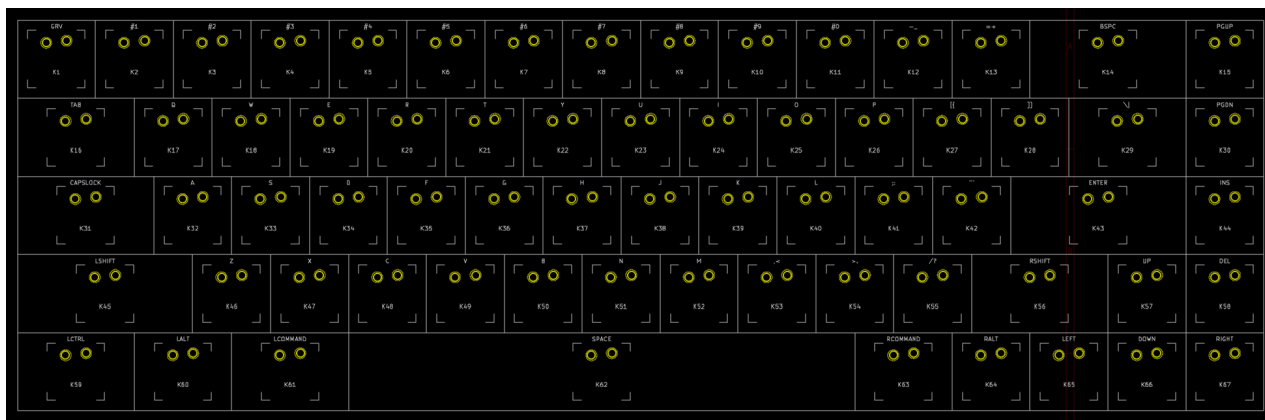
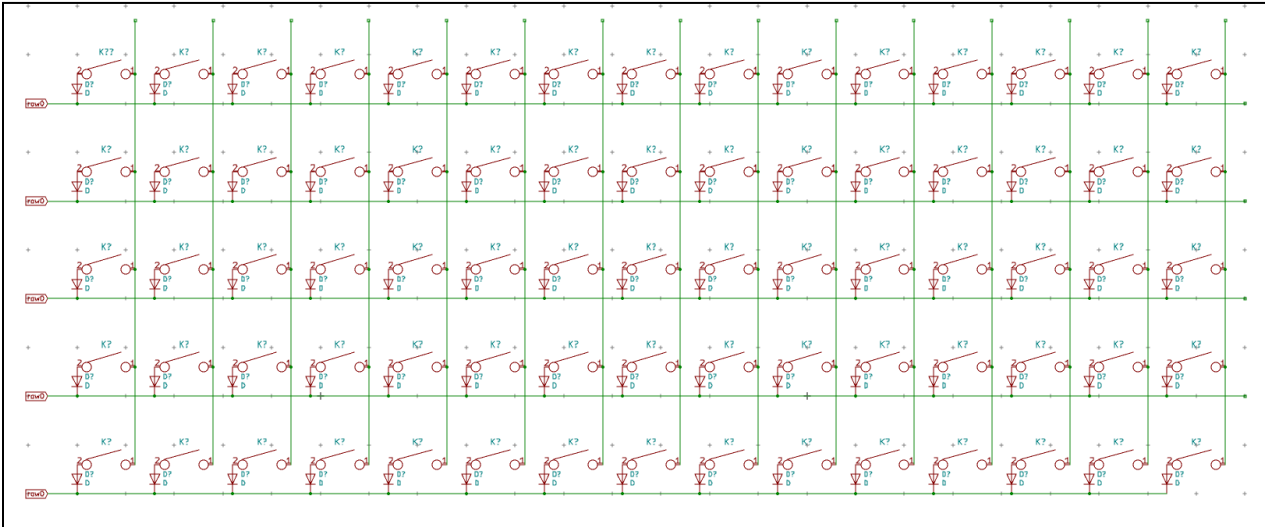
x/x/26

Research: Keyboard PCB

Designed By: Kirk Brian Ventura

Date: x/x/26

Goal:



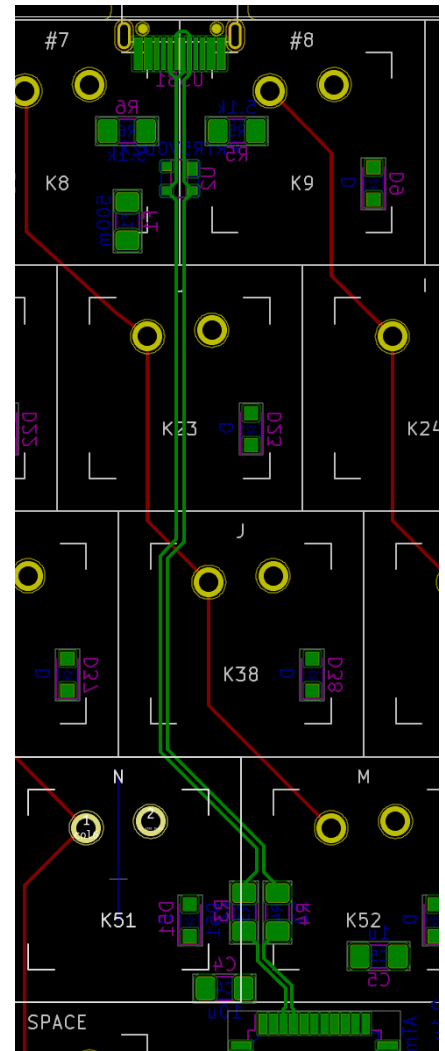
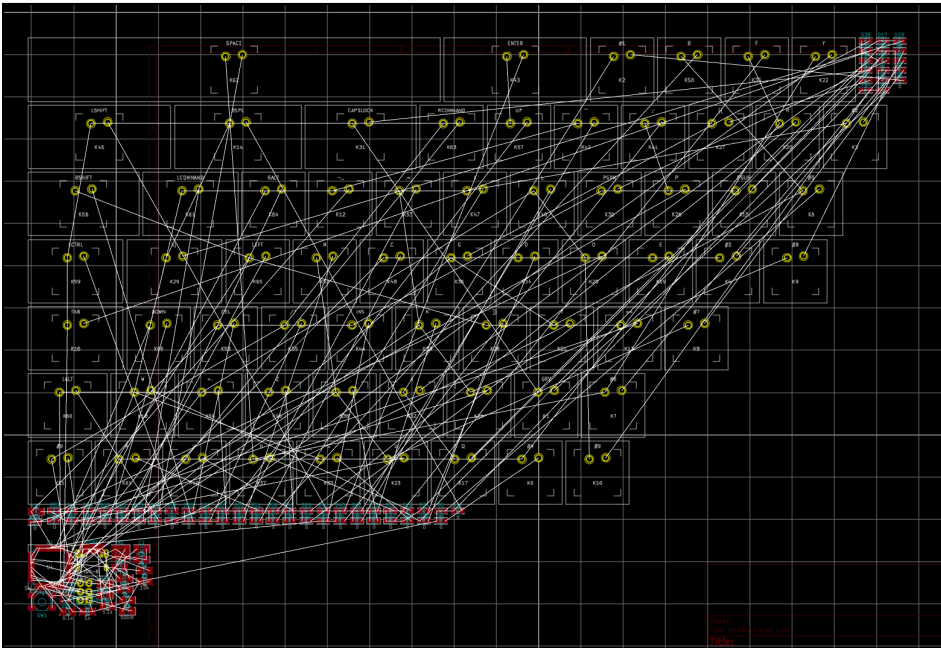
x/x/26

Research: Keyboard PCB

Designed By: Kirk Brian Ventura

Date: x/x/26

Goal:



Appendix: Resources

References

Pg. #	Name	Link
N/A	Excalidraw	https://excalidraw.com/
N/A	Team 2654E Echo Engineering Notebook	https://2654e.netlify.app/nb.pdf
N/A	OnShape	https://www.onshape.com/en/
N/A	KiCAD	https://www.kicad.org/
N/A	Keyboard Article	https://www.masterzen.fr/2020/05/03/designing-a-keyboard-part-1/
	Keyboards	https://ai03.com/
	Schematics	https://adamws.github.io/keyboard-pcbs/
	SVG	https://adamws.github.io/keyboard-pcbs/output/1upkeyboards/pi60_rgb/pi60_rgb-schematic.svg